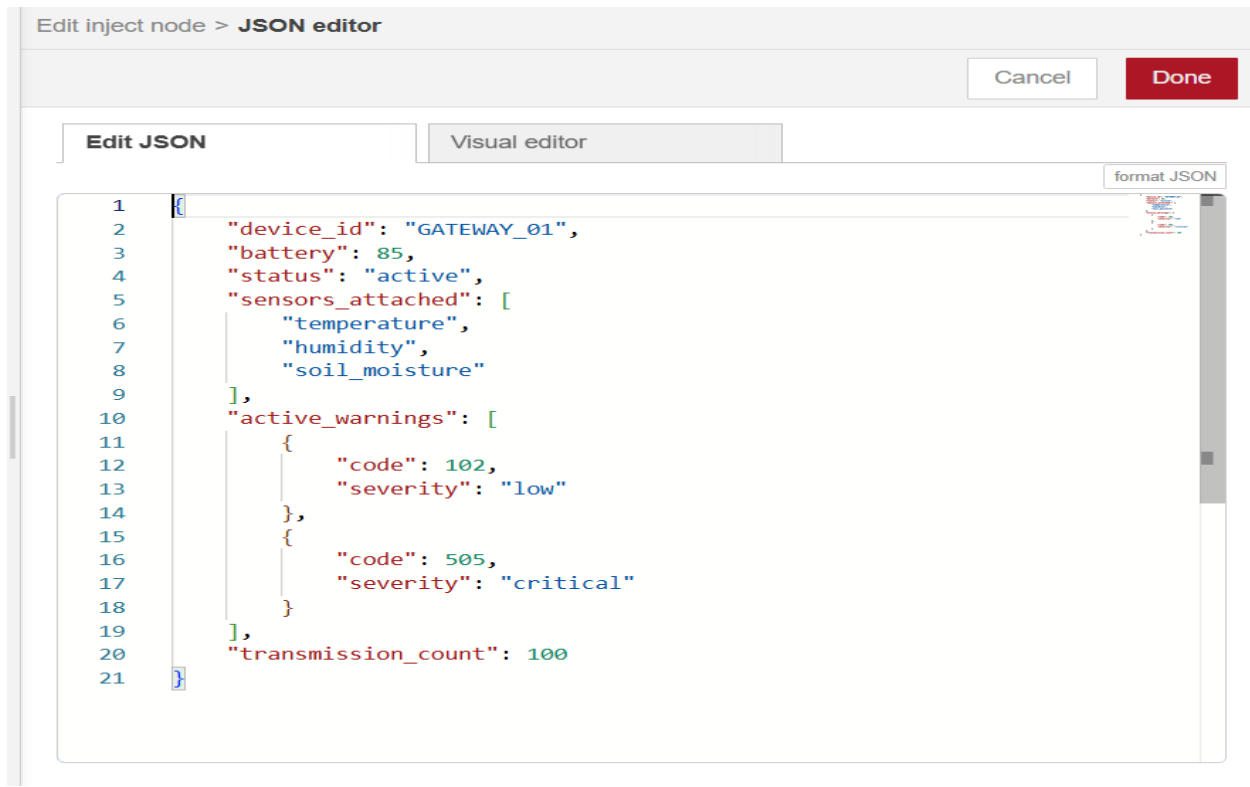


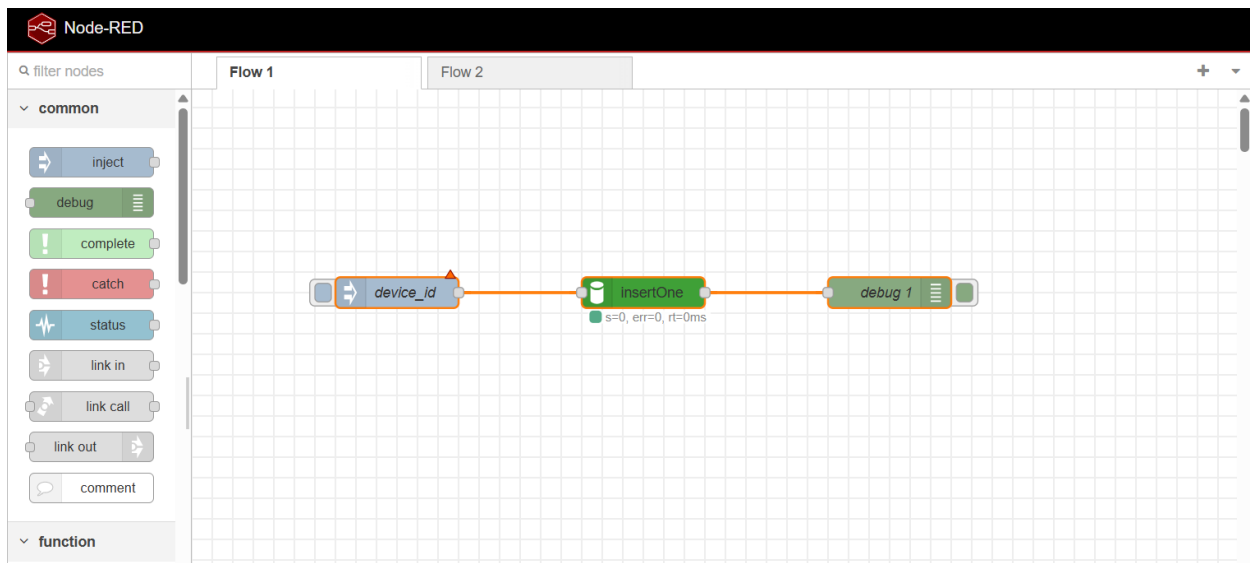
EXERCISE 7

STEP 1

Created the Inject Node

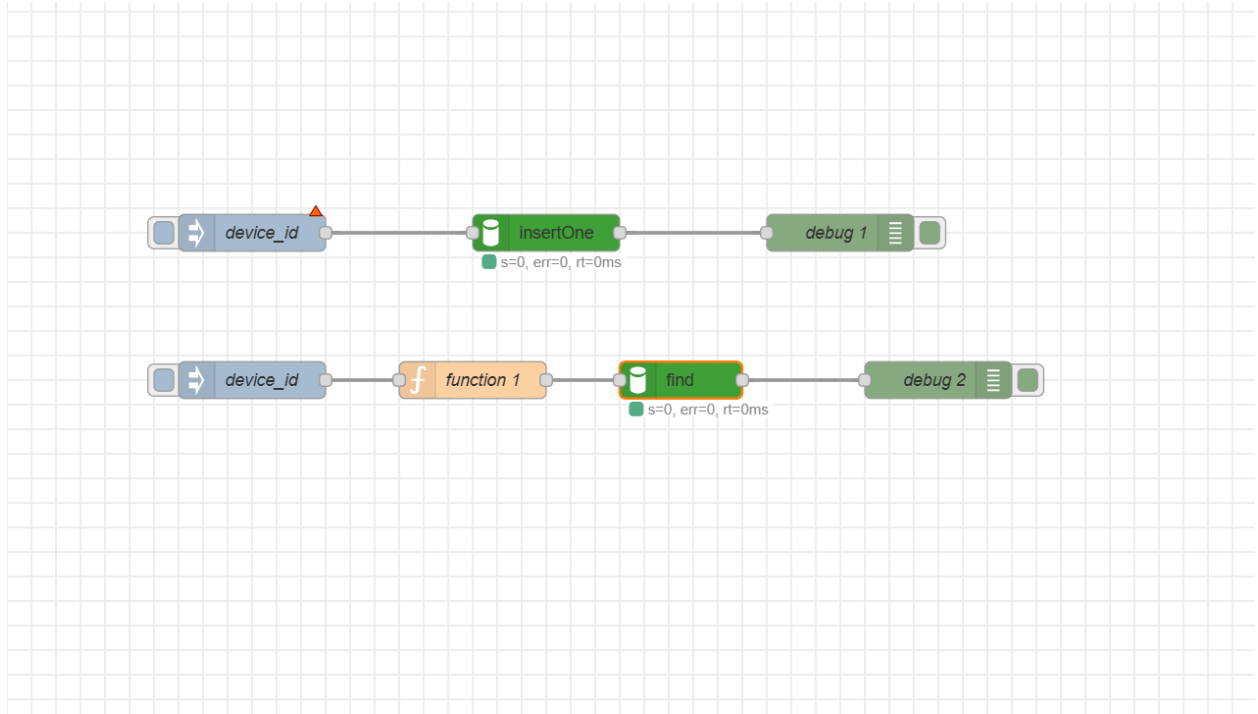


Executed Insertion

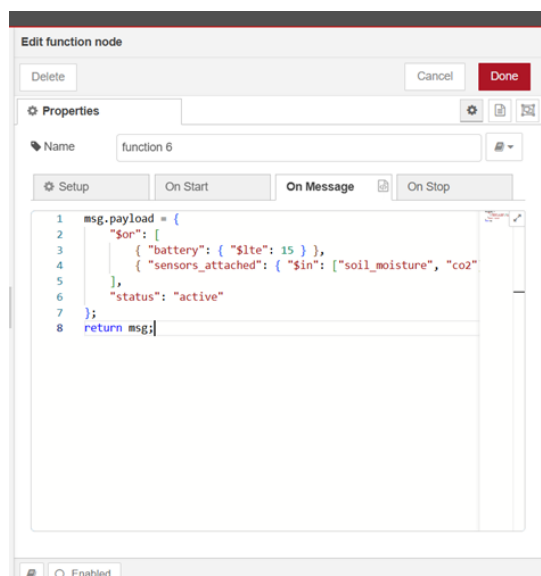


STEP 2

Comparison Operators



A query using \$or to find devices that either have a critical battery OR contain specific sensors in their array using \$in

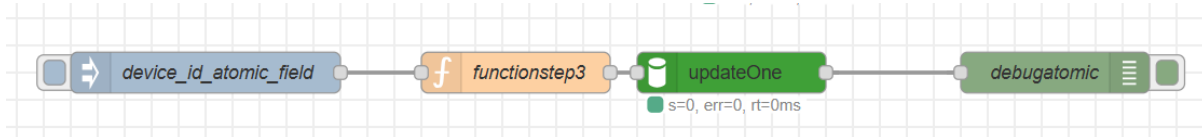


GATEWAY_01 document is successfully returned based on the mathematical logic.

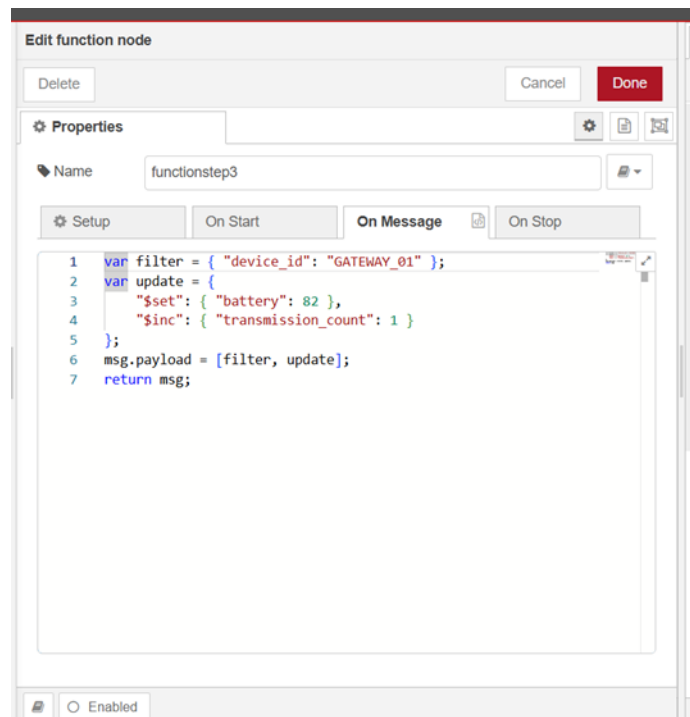
```
msg.payload : array[1]
  ▼ array[1]
    ▼ 0: object
      _id: "6a18240f900f9b9674a7cd2f"
      device_id: "GATEWAY_01"
      battery: 85
      status: "active"
      ▼ sensors_attached: array[3]
        0: "temperature"
        1: "humidity"
        2: "soil_moisture"
      ▼ active_warnings: array[2]
        ▼ 0: object
          code: 102
          severity: "low"
        ▼ 1: object
          code: 505
          severity: "critical"
      transmission_count: 100
```

STEP 3

Updating and Incrementing (\$set, \$inc):



Triggered the flow to find node to verify the transmission_count which is now exactly 101.

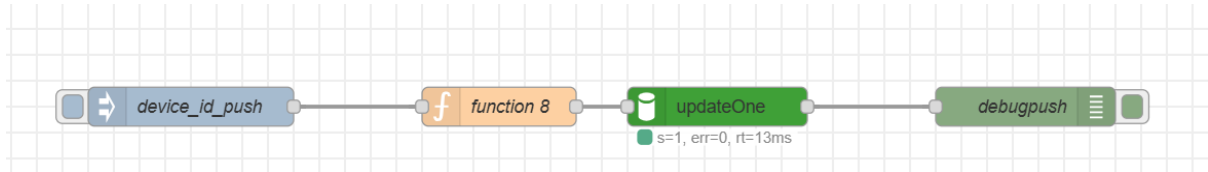


```
msg.payload : Object
  ▼ object
    acknowledged: true
    modifiedCount: 1
    upsertedId: null
    upsertedCount: 0
    matchedCount: 1
```

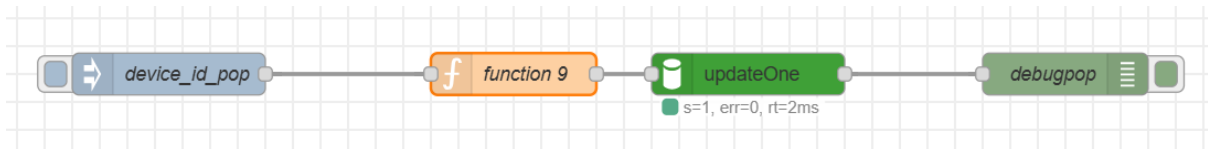
STEP 4

Complex Array Manipulation

Pushing and Popping (\$push, \$pop):

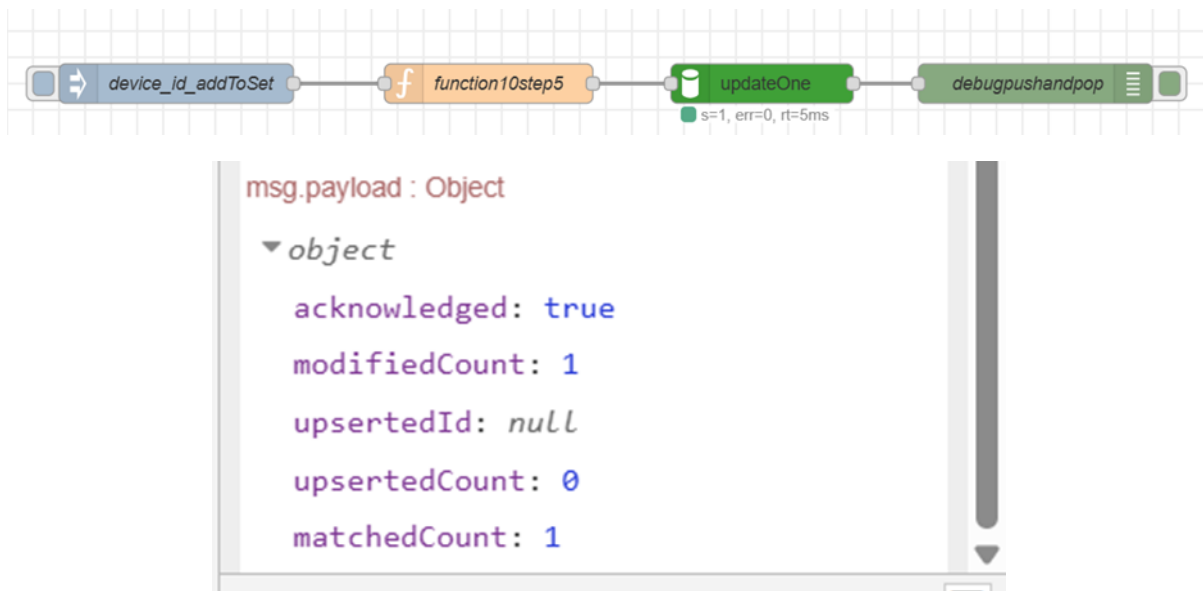


```
msg.payload : Object
  ▼ object
    acknowledged: true
    modifiedCount: 1
    upsertedId: null
    upsertedCount: 0
    matchedCount: 1
```

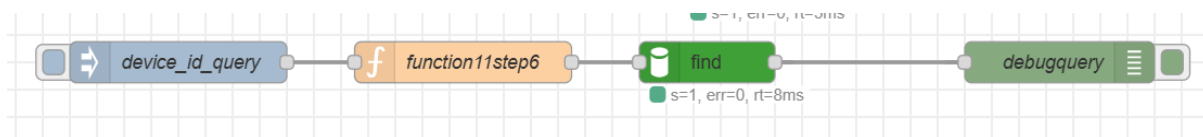


```
msg.payload : Object
  ▼ object
    acknowledged: true
    modifiedCount: 1
    upsertedId: null
    upsertedCount: 0
    matchedCount: 1
```

Smart Set Addition and Removal (\$addToSet, \$pull):



Querying Complex Object Arrays (\$elemMatch):



```
msg.payload : array[1]
  ▼ array[1]
    ▼ 0: object
      _id: "6a18240f900f9b9674a7cd2f"
      device_id: "GATEWAY_01"
      battery: 82
      status: "active"
      ▼ sensors_attached: array[4]
        0: "temperature"
        1: "humidity"
        2: "soil_moisture"
        3: "light_level"
      ▼ active_warnings: array[1]
        ▼ 0: object
          code: 505
          severity: "critical"
          transmission_count: 101
```

Part 5: Discussion & Architectural Analysis

Question 1: The \$inc vs. \$set Paradigm

In Step 3, you used the \$inc operator to increase the transmission count by 1. Why is it architecturally safer to use \$inc: {"count": 1} natively in the database, rather than querying the document in Node-RED, adding 1 to the count in Javascript, and using \$set to write the new total back to the database?

Using \$inc is atomic - it guarantees the operation completes without interference. The alternative approach (read → modify → write) creates a race condition:

APPROACH	RISK
Read → Modify → Write	If two requests read the same value (e.g., 100) at the same time, both add 1, and both write back 101. You lose one increment.
\$inc	The database handles the increment atomically. Even with 1000 concurrent requests, each increment is applied sequentially, so the final count will be 1000.

Question 2: Efficient Payload Management

In Step 4, you utilized \$addToSet and \$pull to manage arrays inside the database. Describe how utilizing these native database array operators significantly reduces the network bandwidth (TCP/IP payload size) between your Node-RED middleware and your MongoDB server compared to rewriting the entire array on every update.

METHOD	WHAT GETS SENT OVER NETWORK	BANDWIDTH
Read → Modify entire array → Write	The entire array (e.g., 10 KB) is downloaded, modified, and re-uploaded.	20 KB per operation
\$addToSet / \$pull	Only the change instruction (e.g., {“\$addToSet”: {“sensors”: “light”} } - about 50 bytes) is sent.	~50 bytes per operation

For an IoT gateway with 100+ sensors in an array, native operators reduce bandwidth by **99.9%** . This is critical for:

- Edge devices with limited bandwidth
- Cellular/LoRaWAN networks (cost per byte)
- High-frequency updates (every second)